

OBJECT ORIENTED SYSTEMS (OOS) — 2023

QUESTION 5 COMPLETE DETAILED SOLUTION

Introduction

Question 5 of the 2023 Object Oriented Systems examination belongs to CO5 and focuses on one of the most advanced and practically important topics in software engineering:

Design Patterns

Design patterns are standardized reusable solutions to recurring software design problems. They help software engineers build:

- scalable systems,
- loosely coupled architectures,
- reusable components,
- and maintainable enterprise applications.

Modern frameworks and large-scale software systems heavily depend upon design patterns. Technologies such as Spring Framework, Java Collections Framework, Android Architecture, GUI systems, Hibernate, and distributed systems all internally use various design patterns  .

This question mainly discusses:

- **Factory Design Pattern**
- **Observer Design Pattern**
- and core software design principles.

Understanding these patterns deeply is extremely important for university examinations, software architecture, and technical interviews.

QUESTION 5(a)

 **Explain Factory Design Pattern with suitable examples.**

 **Answer**

The **Factory Design Pattern** is one of the most important creational design patterns in Object-Oriented Programming. The primary objective of this pattern is to:

- create objects,
- hide object creation logic,
- and provide centralized object management.

Instead of directly creating objects using the **new** keyword everywhere, the Factory Pattern delegates object creation responsibility to a special factory class. This improves loose coupling, flexibility, maintainability, and scalability .

☀ Why Factory Pattern is Needed?

Suppose an application creates different types of vehicles: Car, Bike, Truck. Without the factory pattern:

```
Car c = new Car();  
Bike b = new Bike();
```

Object creation becomes scattered throughout the application. This creates several problems: tight coupling, difficult maintenance, hardcoded object creation, and poor extensibility ⚠.

The Factory Pattern solves this problem by centralizing object creation.

☀ Real-World Analogy

Consider a restaurant 🍴. Customers order food from the waiter, not directly from the kitchen staff. The waiter acts like a factory. The customer only specifies *what is needed*, while the waiter internally arranges *how it is created*. Similarly, the Factory Pattern separates object usage from object creation 💡.

☀ Structure of Factory Pattern

The Factory Pattern generally contains:

Component	Responsibility
Product Interface	Common behavior
Concrete Products	Actual implementations
Factory Class	Creates objects
Client	Uses objects

☀ Step-by-Step Working

The Factory Pattern works as follows:

1. Client requests an object from the factory.
2. Factory decides which object to create.
3. Factory returns the object reference.
4. Client uses the object without knowing creation details.

Thus object creation becomes abstracted.

✅ Java Program for Factory Pattern

```
interface Shape {  
    void draw();  
}  
  
class Circle implements Shape {
```

```
    public void draw() {
        System.out.println("Drawing Circle");
    }
}

class Rectangle implements Shape {
    public void draw() {
        System.out.println("Drawing Rectangle");
    }
}

class ShapeFactory {
    Shape getShape(String type) {
        if(type.equals("Circle")) {
            return new Circle();
        } else if(type.equals("Rectangle")) {
            return new Rectangle();
        }
        return null;
    }
}

class Main {
    public static void main(String args[]) {
        ShapeFactory factory = new ShapeFactory();

        Shape s1 = factory.getShape("Circle");
        s1.draw();

        Shape s2 = factory.getShape("Rectangle");
        s2.draw();
    }
}
```

Output

```
Drawing Circle
Drawing Rectangle
```

🌟 Deep Explanation of Program

- Initially, the **Shape** interface defines common behavior.
- Then, **Circle** and **Rectangle** implement **Shape**.
- The factory class **ShapeFactory** contains object creation logic.
- Instead of the client directly creating objects, the factory creates appropriate objects dynamically.

Thus the client remains independent from concrete class implementation 🚀.

🌟 Advantages of Factory Pattern

The Factory Pattern provides several important advantages:

1. It promotes loose coupling because client code depends on abstraction rather than concrete classes.
2. Object creation logic becomes centralized.
3. Adding new object types becomes easier.
4. Maintainability and scalability improve significantly.

⚠ Disadvantages

The Factory Pattern may increase the number of classes, introduce abstraction complexity, and slightly increase design overhead. However, benefits generally outweigh disadvantages in large systems.

☀ Real-World Applications

The Factory Pattern is widely used in:

- JDBC driver loading
- Spring Bean Factory
- GUI frameworks
- XML parsers
- Object pooling systems

For example, the Java `Calendar` class internally uses factory methods.

☀ Difference Between Direct Object Creation and Factory Pattern

Direct Creation	Factory Pattern
Tight coupling	Loose coupling
Scattered object creation	Centralized creation
Hard to maintain	Easier maintenance
Less flexible	Highly extensible

🎯 Conclusion

The Factory Design Pattern centralizes object creation and promotes loose coupling, scalability, and maintainability. It is one of the most widely used creational patterns in enterprise software development 🚀.

📄 QUESTION 5(b)

❓ **Explain Observer Design Pattern with suitable examples.**

✅ Answer

The **Observer Design Pattern** is one of the most important behavioral design patterns in Object-Oriented Programming. It establishes a one-to-many dependency relationship between objects.

Whenever one object changes state, all dependent objects automatically receive notification. This pattern is extremely useful in event-driven systems, GUI applications, real-time systems, and distributed architectures



☀ Why Observer Pattern is Needed?

Suppose a stock market application has multiple investors, dashboards, mobile apps, and notification systems. Whenever the stock price changes, all observers must receive an update automatically.

Without the observer pattern, tight coupling occurs, notification logic becomes complex, and maintainability decreases ⚠. The Observer Pattern solves this elegantly.

☀ Real-World Analogy

Consider a YouTube channel 📺. When a creator uploads a video, subscribers automatically receive a notification. Here:

- Channel → **Subject**
- Subscribers → **Observers**

This perfectly represents the Observer Pattern 💡.

☀ Components of Observer Pattern

Component	Responsibility
Subject	Maintains observers
Observer	Receives updates
ConcreteSubject	Actual state holder
ConcreteObserver	Actual observer

☀ Working Principle

The Observer Pattern works as follows:

1. Observers register with the subject.
2. Subject changes state.
3. Subject notifies all observers.
4. Observers update automatically.

Thus dependency management becomes dynamic.

✅ Java Program for Observer Pattern

```
import java.util.*;

interface Observer {
    void update(int value);
}
```

```
class Subscriber implements Observer {
    String name;

    Subscriber(String name) {
        this.name = name;
    }

    public void update(int value) {
        System.out.println(name + " received update: " + value);
    }
}

class Channel {
    private List<Observer> observers = new ArrayList<>();
    private int data;

    void subscribe(Observer o) {
        observers.add(o);
    }

    void setData(int value) {
        data = value;
        notifyObservers();
    }

    void notifyObservers() {
        for(Observer o : observers) {
            o.update(data);
        }
    }
}

class Main {
    public static void main(String args[]) {
        Channel c = new Channel();

        Subscriber s1 = new Subscriber("A");
        Subscriber s2 = new Subscriber("B");

        c.subscribe(s1);
        c.subscribe(s2);

        c.setData(100);
    }
}
```

Output

```
A received update: 100
B received update: 100
```

🌟 Deep Explanation of Program

- The interface **Observer** defines the update mechanism. Each **Subscriber** implements the **update()** method.
- The class **Channel** acts as the **Subject**.
- Whenever **setData()** changes state, all observers automatically receive notification.

Thus one-to-many dependency becomes automated 🚀.

🌟 Advantages of Observer Pattern

The Observer Pattern provides several major advantages:

1. It promotes loose coupling between the subject and observers.
2. Dynamic subscription becomes possible.
3. Real-time update systems become easier.
4. Scalability and flexibility improve significantly.

⚠️ Disadvantages

The Observer Pattern may increase notification overhead, create cascading updates, and complicate debugging in very large systems. Thus proper design is important.

🌟 Real-World Applications

The Observer Pattern is widely used in:

- GUI event handling
- Stock market systems
- Weather applications
- Notification systems
- Social media feeds
- MVC architectures

For example, Java Swing event listeners internally follow the Observer Pattern.

🌟 Difference Between Direct Dependency and Observer Pattern

Direct Dependency	Observer Pattern
Tight coupling	Loose coupling
Hardcoded communication	Dynamic notification
Poor scalability	Highly scalable
Difficult maintenance	Easier maintenance

🌟 JVM Perspective

During runtime, observer references are stored dynamically, notifications are propagated automatically, and callback methods are executed sequentially. Thus event-driven programming becomes possible 🚀.

🎯 Conclusion

The Observer Design Pattern establishes dynamic one-to-many relationships between objects and enables automatic notification propagation. It is one of the most important behavioral design patterns in modern software engineering 🚀.